Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

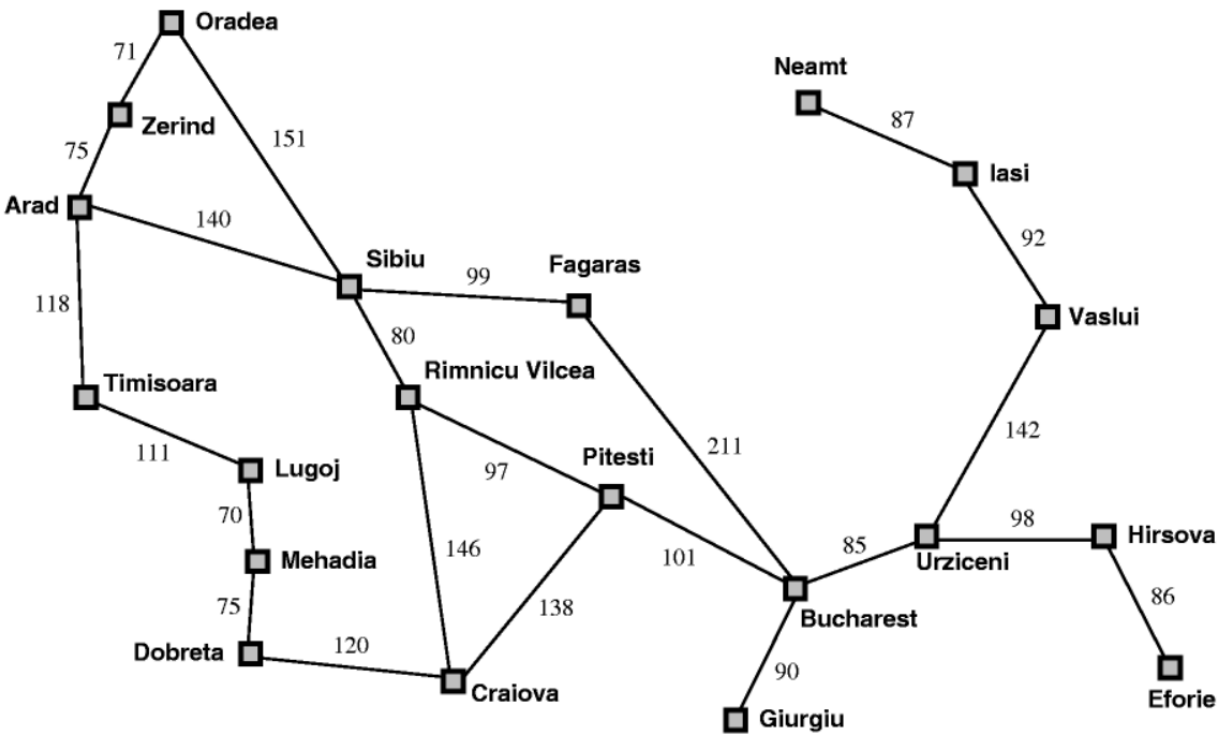
MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan III

Studi Kasus

Dalam studi kasus kali ini, akan melakukan pencarian jarak terdekat dari kota Arad ke Bucharest.




```
# Define graph with tipe data dictionary
graph = {
  'A' : ['Z','S', 'T'],
  'Z' : ['O'],
  'O' : ['S1'],
  'S1' : ['F','R'],
  'F' : ['B'],
  'B' : [],
  'R' : [],
  'S' : ['O1', 'F1', 'R1'],
  'O1' : [],
```

```
  'F1' : ['B1'],
  'B1' : [],
  'R1' : ['C', 'P'],
  'C' : [],
  'P' : [],
  'T' : ['L'],
  'L' : ['M'],
  'M' : ['D'],
  'D' : []
}
```

```
visited = set() # Set to keep track of visited nodes of graph.
```

```
def dfs(visited, graph, node):  # Function for dfs
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

```
print("Following is the Depth-First Search")
```

```
Following is the Depth-First Search
```

```
dfs(visited, graph, 'A')
```

```
A
Z
O
S1
F
B
R
S
O1
F1
B1
R1
C
P
T
L
M
D
```

Logika Pemrograman: Algoritma DFS (Depth-First Search) — Studi Kasus Graf Kota

Mata Kuliah: Praktikum Terapan Teori Graf — Universitas Gunadarma **Pertemuan III**

Studi Kasus

Tujuan dari program ini adalah melakukan pencarian jalur dari kota **Arad** ke **Bucharest** menggunakan metode **Depth-First Search (DFS)** pada sebuah graf yang merepresentasikan peta jalan antar kota.

Langkah 1 — Mendefinisikan Graf

Pertama-tama, dibuatlah sebuah struktur data berupa **dictionary** (kamus) Python untuk merepresentasikan graf. Setiap kunci (*key*) adalah nama sebuah simpul/kota, dan nilainya (*value*) adalah daftar kota-kota yang bertetangga langsung (terhubung langsung).

```
graph = {
  'A'  : ['Z', 'S', 'T'],
  'Z'  : ['O'],
  'O'  : ['S1'],
  'S1' : ['F', 'R'],
  'F'  : ['B'],
  'B'  : [],
  'R'  : [],
  'S'  : ['O1', 'F1', 'R1'],
  'O1' : [],
  'F1' : ['B1'],
  'B1' : [],
  'R1' : ['C', 'P'],
  'C'  : [],
  'P'  : [],
  'T'  : ['L'],
  'L'  : ['M'],
  'M'  : ['D'],
  'D'  : []
}
```

Artinya:

- Kota `A` terhubung ke `Z`, `S`, dan `T`
- Kota `Z` hanya terhubung ke `O`
- Kota `B`, `R`, `O1`, `B1`, `C`, `P`, dan `D` adalah simpul akhir (tidak punya tetangga), artinya ini adalah **daun (leaf node)** dalam graf

---

Langkah 2 — Menyiapkan Pelacak Simpul yang Sudah Dikunjungi

Sebelum penelusuran dimulai, dibuatlah sebuah **set** kosong bernama `visited`. Fungsinya adalah mencatat simpul mana saja yang sudah pernah dikunjungi selama proses DFS berlangsung.

```
visited = set()
```

Kenapa pakai `set`? Karena `set` secara otomatis menghindari duplikat — setiap simpul hanya akan muncul sekali meski didaftarkan berkali-kali.

Langkah 3 — Membuat Fungsi DFS (Rekursif)

Inti dari program ini ada di fungsi `dfs()` yang bersifat **rekursif** (memanggil dirinya sendiri).

```
def dfs(visited, graph, node):
    if node not in visited:
        print(node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

Logika di dalam fungsi ini:

1. **Cek apakah simpul sudah dikunjungi** — Jika `node` belum ada di dalam `visited`, maka lanjut ke langkah berikutnya. Jika sudah ada, lewati saja (tidak diproses ulang).
2. **Cetak nama simpul** — Simpul yang baru dikunjungi ditampilkan ke layar sebagai bukti bahwa penelusuran sedang melewati simpul tersebut.
3. **Tandai sebagai sudah dikunjungi** — Simpul dimasukkan ke dalam `visited` agar tidak dikunjungi lagi.
4. **Kunjungi tetangga satu per satu** — Untuk setiap tetangga (`neighbour`) dari simpul saat ini, fungsi `dfs()` dipanggil lagi secara rekursif. Ini yang membuat DFS menelusuri **sedalam mungkin** ke satu cabang sebelum berpindah ke cabang lain.

Langkah 4 — Menjalankan Penelusuran

Penelusuran dimulai dari simpul `'A'` (merepresentasikan kota Arad):

```
print("Following is the Depth-First Search")
dfs(visited, graph, 'A')
```

Alur Penelusuran (Trace Eksekusi)

Berikut adalah urutan simpul yang akan dikunjungi beserta penjelasan logikanya:

Urutan	Simpul Dikunjungi	Penjelasan
1	A	Titik awal. Punya tetangga: Z, S, T
2	Z	Tetangga pertama A. Punya tetangga: O
3	O	Tetangga dari Z. Punya tetangga: S1
4	S1	Tetangga dari O. Punya tetangga: F, R
5	F	Tetangga pertama S1. Punya tetangga: B
6	B	Daun. Tidak punya tetangga, rekursi berhenti
7	R	Tetangga kedua S1. Daun, rekursi berhenti
8	S	Kembali ke tetangga kedua A. Punya tetangga: O1, F1, R1
9	O1	Daun. Rekursi berhenti
10	F1	Punya tetangga: B1
11	B1	Daun. Rekursi berhenti
12	R1	Punya tetangga: C, P
13	C	Daun. Rekursi berhenti
14	P	Daun. Rekursi berhenti
15	T	Tetangga ketiga A. Punya tetangga: L
16	L	Punya tetangga: M
17	M	Punya tetangga: D
18	D	Daun. Rekursi berhenti. DFS selesai

Konsep Kunci yang Dipakai

- **DFS (Depth-First Search)** → Strategi penelusuran yang menelusuri satu cabang **sedalam mungkin** sebelum berpindah ke cabang lain. Berbeda dengan BFS yang menelusuri secara melebar level per level.
- **Rekursi** → Fungsi memanggil dirinya sendiri untuk meneruskan penelusuran ke simpul berikutnya.
- **Set** `visited` → Mencegah simpul dikunjungi lebih dari sekali, sekaligus menghindari *infinite loop* pada graf yang mengandung siklus.
- **Dictionary sebagai representasi graf** → Cara yang efisien untuk menyimpan daftar ketetanggaan (*adjacency list*) dalam Python.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/